



Inspiring Excellence

## BRAC UNIVERSITY

### CSE360: Computer Interfacing

#### Lab Project Report

**Title:** *Smart Medicine Adherence Box*

by

Group\_3  
Section 5

#### Group Members:

| Name                        | ID       |
|-----------------------------|----------|
| Jotee Sarkar Joy            | 22301001 |
| Subrina Jerrin Akter        | 22201142 |
| Md. Abir Hasan Rohan        | 22101294 |
| Shees Mohammad Hossain Saud | 20101234 |

Department of Computer Science and Engineering  
BRAC University

August 23, 2026

## Abstract

The project named “DoseFlow” is a smart and connected medication reminder and adherence monitoring box for assisting users, particularly elderly and chronically ill, to take their medicines on time without needing to depend upon their memory and manual tracking. The system consists of an Arduino Uno microcontroller and the user interface display driver, and an ESP32 DevKit V1 as the network connected slave to provide Wi-Fi connectivity, cloud sync and time-of-day retrieval. The Arduino drives a finite state machine (Waiting, Due, Snoozed, Confirmed, Missed), a real-time clock (DS3231) and a time-of-flight distance sensor (VL53L0X) to determine if the box lid is open; a capacitive touch sensor for confirming the dose intake and a physical snooze button. The ESP32 has a built-in captive Wi-Fi setup portal on the ESPAsyncWebServer which will require a first-time setup. It periodically fetches the medicine schedule from the production cloud application (dose.joysarkar.app), sends the medicine schedule and current configuration to the Arduino through a UART level shifted interface and writes the adherence events (reminder due, user confirmed, snooze, missed, open, close) into the cloud database (mongodb). If it's not connected to the internet, it will store these logs on the 4MB of non-volatile flash (LittleFS) memory built into the ESP32. Powered by a 3x 18650 rechargeable battery pack which is regulated by a TP4056 charge/protection board and MT3608 boost converter. The final outcome is a low-cost, low-power, repeatable embedded system that brings the physical reminder box together with a medication schedule in the cloud minimizing medication errors and providing caregivers with adherence visibility.

## Keywords

Arduino, ESP32, Internet of Things (IoT), medicine adherence, embedded systems, MongoDB, real-time clock, time-of-flight sensing

## 1. Introduction

**Problem and Motivation:** The problem and motivation is that missed or mistimed medication doses happen frequently and are a serious issue particularly in elderly users, chronic medicine users and any person that has to take multiple doses throughout the day. A paper chart and/or a telephone alarm are not sufficient to use memory. An alarm on the phone could be ignored and it would be impossible to ascertain whether the medicine box had actually been physically opened or not. When administering medicine from a distance, it can be difficult for caregivers to ensure the medication was taken and that it was not “snoozed” or missed. Therefore, there is a requirement to have a low cost reminder system which can notify the user, confirm what happened and record it.

**Objective:** This project aims to develop a connected medicine box that reliably reminds the user at the programmed time, that detects if the user has opened the box and requires a physical action (opening box and touch confirmation) to acknowledge that he/she has taken the medicine, has one single controlled snooze feature within a set slot of time after the dose has been taken and can keep the box's internal clock correct through internet synchronization, and maintains synchronization with the medicine schedule and log recorded by the user to a MongoDB cloud-based backend so that the data can be accessed even if the box loses power or fails to connect with the internet.

**Scope & Relevance:** The project is based on the two-microcontroller system, where one Arduino Uno microcontroller is programmed to act as an on-device logic & UI controller and the other ESP32 DevKit V1 is used as the network gateway. The system will recognize a fixed set of physical events: lid open/close, touch confirmation and snooze press and will control a fixed five state adherence state machine (Waiting, Due, Snooze, Confirmation, Missed). Schedule and configuration pulled from a cloud-based external application, and in the absence of any network credentials, a local self-hosted captive portal is used. Concerning computer interfacing courses, the project is very relevant as it deals with two ways that hardware communicate with each other: I2C and level shifted UART, and secondly, a hardware interrupt-style event handling for a physical state machine and hardware-to-cloud communication using the ESP32 as a network bridge for the Arduino.

## 2. Related Work/ Inspiration

Noncompliance with prescription medications has been well documented as a public health issue. World Health Organisation estimates that approximately 50% of individuals with chronic diseases don't take the medicines as prescribed, and that this is 'a prime cause of treatment failure and avoidable hospitalisations' (World Health Organisation, 2003). This creates a wider spectrum of smart drug-adherence devices that would rely on a more tangible reminder/dispensing device that could be automated and have built-in alerts and adherence logs to reduce the need for memory.

The nearest technical forerunner of DoseFlow is Satwani (2026) which utilizes a similar sensing stack of an ESP32 microcontroller, a DS3231 RTC (Real-Time Clock) over an I2C interface, an OLED display, and a buzzer alert. By the authors' own admission this is a single-microcontroller, button-programmed machine that reliably tracks time and gives you an audible/visual warning but doesn't prove that you actually took the medicine, something that the authors acknowledge is for future work. DoseFlow instead of one has two microcontrollers: one for local, time-critical control, and another for Wi-Fi, cloud sync and remote scheduling; and a second layer of confirmation: a capacitive touch sensor for a definite confirmation and a VL53L0X time-of-flight sensor for a really open lid.

To take adherence verification one step further, Peddisetti et al. (2024) propose a Smart Pill Dispenser and Smart Cup system featuring an IMU and ultrasonic sensor inside the cup that sends a signal to a companion Android application via Bluetooth Low Energy to verify adherence. They determined that the motion-based consumption verification is very accurate, but the extra complexity of the dispensing device and second wearable device and BLE-to-phone link makes the system's total cost and complexity far more expensive and complex than a box that is sitting on the ground. Both of those previous systems depend on the patient using an app that connects to a phone, while DoseFlow offers an adherence log and the entire adherence schedule to be synced to a MongoDB database that can be used to explore the past without the patient's device.

The individual elements of DoseFlow such as the limit switches, the scheduling by RTC, buzzer alerts, and Wi-Fi logging are all features found in previous medicine-box designs including the above surveyed generic limit-switch/RFID box. The pieces are assembled in a special way that makes DoseFlow unique. MediMind (Satwani, 2026) has a very similar sensing stack, but as they say, they don't confirm that a dose was taken. DoseFlow does that, and in particular disallows any implicit touch confirmation. The authors relied solely on commodity parts, and a mechanical limit switch to act as a cross-check of the VL53L0X

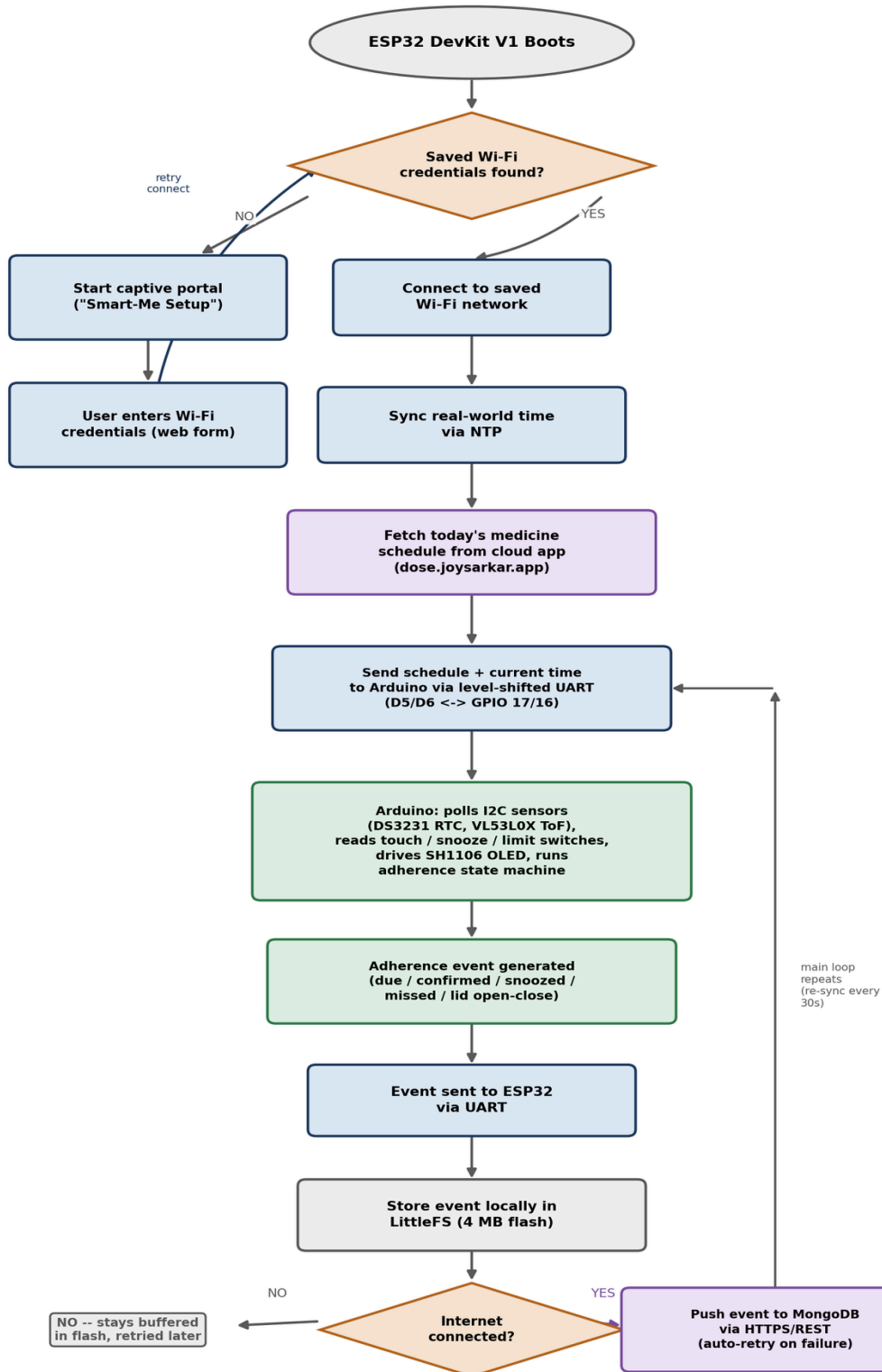
time of flight reading. The motion sensing Smart Cup and pill dispenser developed by Peddisetti et al., (2024) gives DoseFlow even higher confirmation accuracy, but is significantly more complex and expensive, requiring a second device and some moving parts, and a companion phone app to connect to via Bluetooth. It also uses two microcontrollers instead of one, so the alarm loop doesn't get slow if the ESP32 has slow network or Wi-Fi communications, which isn't an issue for the single-MCU designs, such as MediMind, but could be if they decide to add network functionality into their design. Last but not least, unlike the former systems, the adherence and the entire schedule are stored in a MongoDB database and DoseFlow is installed using a local self-hosted WiFi captive portal, thus allowing caregivers to track adherence remotely and elderly users to not have to own or configure a phone.

### 3. Technical Approach

**System Architecture:** Dual-microcontroller script driven architecture which enables to separate real time local control and network connectivity. The "logic master" is the Arduino Uno which reads the touch sensor, the snooze button and the two limit switches directly, polls the VL53L0X time-of-flight sensor and DS3231 RTC over I2C, drives the 1.3" SH1106 OLED display and runs the adherence state machine. These are the "network slave" - after ESP32 DevKit V1 is booted, it will either connect to a saved WiFi network or fall back to an ESPAsyncWebServer captive portal ("Smart-Me Setup" - a user enters his or her credentials on it). When it is connected it synchronizes with the real-world time via NTP, calls a medicine schedule for the day from the cloud application (`dose.joysarkar.app`) and then sends this schedule and the current time to the Arduino via UART communication. The events generated by the Arduino (reminder due, dose confirmed, snoozed, missed, lid opened/closed) are pushed back into esp32 and stored locally in the ESP32's 4MB of built-in non-volatile flash (LittleFS) and then sent to the MongoDB database, with auto-retry if the connection is restored.

The two pins are dedicated for communication (Arduino D5/Arduino D6 to ESP32 GPIO 17/ESP32 GPIO 16) and the signal is safely stepped down by the Arduino UART signal to the logic level of the ESP32 using a resistor voltage divider. With the peripherals, the OLED display, RTC module and time-of-flight sensor are all connected to a single I2C bus (SDA/SCL) on the Arduino which helps to keep the wiring spaghetti ball to a manageable size.

## DoseFlow -- System Architecture Workflow



## Components Used:

### Hardware:

| #  | Component                            | Role in the System                                                           |
|----|--------------------------------------|------------------------------------------------------------------------------|
| 1  | Arduino Uno R3                       | Local logic controller, state machine, display driver                        |
| 2  | ESP32 DevKit V1                      | Wi-Fi connectivity, cloud API sync, NTP time sync                            |
| 3  | TP223 Capacitive Touch Sensor        | Confirms the user has physically interacted with the box (dose confirmation) |
| 4  | Snooze Push Button                   | Triggers a single, one-time snooze per dose window                           |
| 5  | Limit Switch × 2                     | Mechanically detects whether the lid is open or closed                       |
| 6  | VL53L0X Time-of-Flight Sensor        | Optically confirms lid-open state, cross-checked against the limit switches  |
| 7  | DS3231 RTC Module                    | Maintains accurate real-time clock for scheduling backed by a CR2032 battery |
| 8  | 1.3" SH1106 OLED Display (I2C)       | Displays current time, next dose, countdown and system state to the user     |
| 9  | Active Buzzer                        | Audible alarm when a dose is due                                             |
| 10 | TP4056 Charge/Protection Module      | Manages charging and protection for the Li-ion battery pack                  |
| 11 | MT3608 Boost Converter               | Steps battery voltage up to a regulated 5V supply                            |
| 12 | 3× 18650 Li-ion Batteries (parallel) | Rechargeable power source for the whole device                               |
| 13 | Resistors (1kΩ / 2kΩ)                | Voltage divider for safe 5V→3.3V UART level shifting                         |
| 14 | Breadboard & Jumper Wires            | Prototyping and interconnects                                                |

### Software:

1. Arduino IDE (C++ firmware for both the Uno and the ESP32)
2. Libraries: SoftwareSerial, U8g2lib (OLED), Wire (I2C), RTCLib (DS3231), VL53L0X, WiFi, WiFiClientSecure, HTTPClient, DNSServer, ESPAsyncWebServer, LittleFS, Preferences, ArduinoJson
3. MongoDB database: connected via a cloud-hosted REST API (schedules, logs and acknowledgement endpoints) for storing the medication schedule and adherence history
4. NTP (Network Time Protocol) for automatic clock synchronization on the ESP32

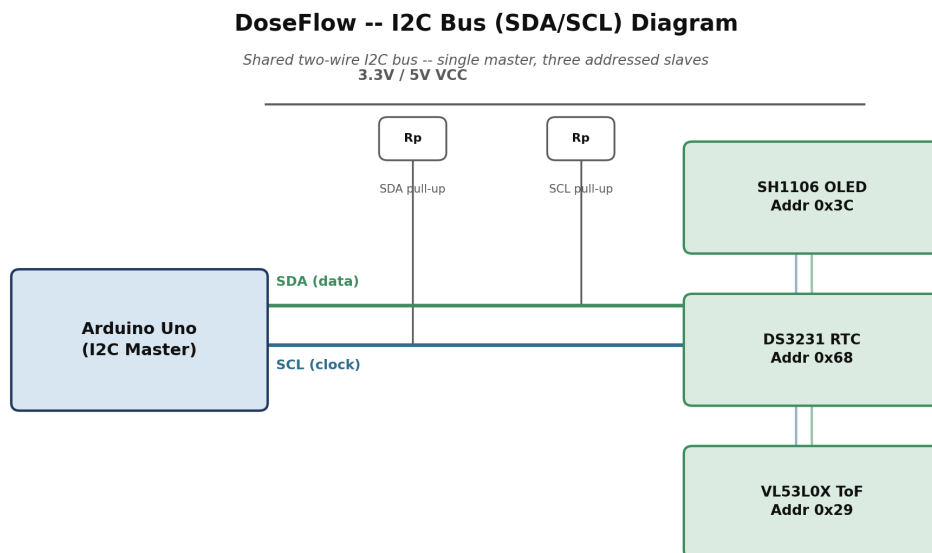
### Cost Breakdown:

| Component                     | Price (BDT) |
|-------------------------------|-------------|
| RFL Ice Box 8L                | 1090        |
| Arduino Uno R3                | 990         |
| ESP32 DevKit V1               | 670         |
| TP223 Capacitive Touch Sensor | 50          |
| Snooze Push Button            | 15          |
| Limit Switch × 2              | 100         |

|                                         |              |
|-----------------------------------------|--------------|
| VL53L0X Time-of-Flight Sensor           | 450          |
| DS3231 RTC Module (with AT24C32 EEPROM) | 320          |
| 1.3" SH1106 OLED Display (I2C)          | 530          |
| Active Buzzer                           | 20           |
| TP4056 Charge/Protection Module         | 50           |
| MT3608 Boost Converter                  | 99           |
| 3× 18650 Li-ion Batteries               | 600          |
| Resistors (1kΩ / 2kΩ, voltage divider)  | 20           |
| Breadboard                              | 140          |
| Jumper Wires                            | 512          |
| Soldering Iron                          | 310          |
| Glue Gun & Glue Sticks                  | 320          |
| Flux & Solder Wire                      | 219          |
| Masking Tape                            | 50           |
| <b>Total</b>                            | <b>6,555</b> |

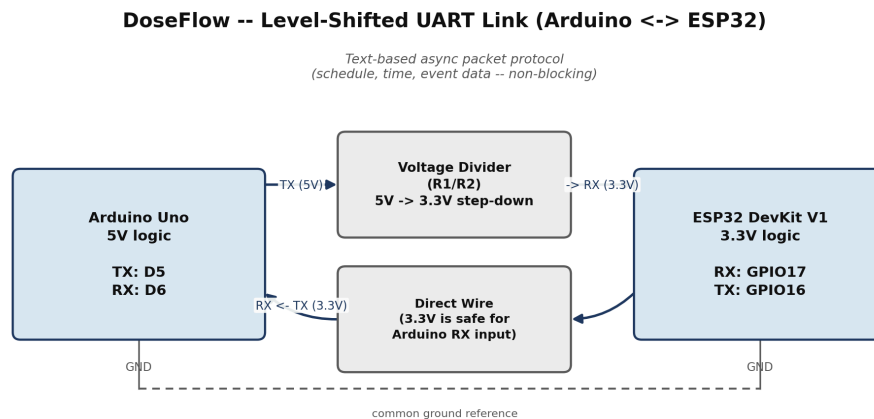
### Communication Protocol:

1. **I2C (Inter-Integrated Circuit):** This is used to communicate with the OLED display, DS3231 RTC and VL53L0X time-of-flight sensor from the Arduino. All three peripherals are digital modules which make their data available via their internal registers without having to perform any analog signal processing on the Arduino. I2C allows all three to use a common two-wire bus (SDA/SCL) each being accessed individually so the wiring is kept to a minimum.



2. **UART (level-shifted serial):** Bidirectional communication with Arduino and ESP32. The Arduino sends a 5V signal to the ESP32 on dedicated pins in the hardware UART2 and the

ESP32 connects to a resistor voltage divider, which drops the voltage to 3.3V before it reaches the ESP32. This is a simple text based packet protocol which enables the exchange of schedule, time and event data between the two microcontrollers in an asynchronous manner with no blocking of each micro controller's main loop.



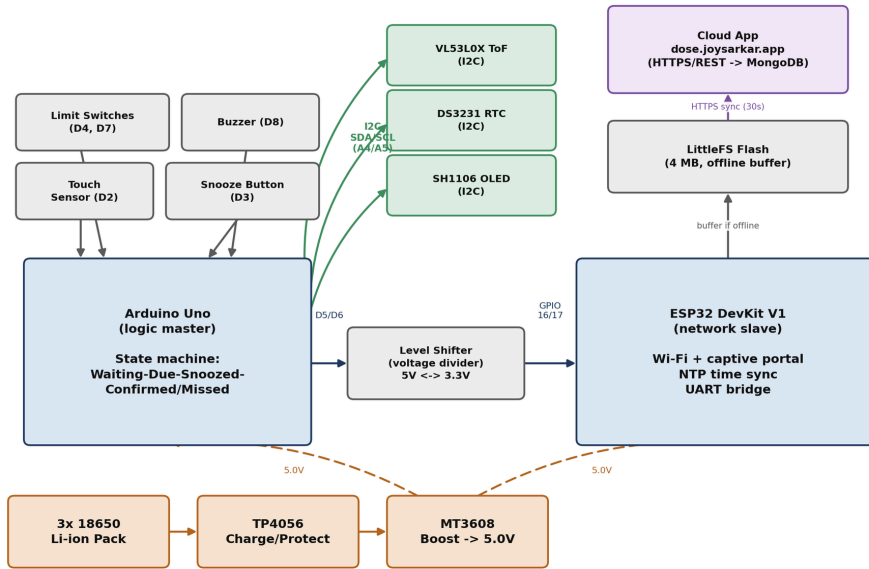
3. **HTTPS/REST (Wi-Fi):** used to communicate with the cloud back-end by the ESP32. The ESP32 repeatedly calls a REST API to get a medicine schedule for the day, and pushes an adherence log to a MongoDB database, using key authentication over HTTPS. This offers local buffering in the ESP32's 4MB flash memory (LittleFS) in case of temporary interruption of the network, while also giving local access and visibility to the schedule and adherence history.

### Circuit/Schematic:

**Physical build:** The touch sensor is connected to the Arduino Uno (D2), the snooze button (D3), the limit switches (D4 and D7), the buzzer (D8), and the UART communication to the ESP32 (D5/D6, level-shifted). The I2C bus (A4/SDA and A5/SCL) is shared by the OLED, RTC and ToF sensors. The output of the boost converter is connected to the 5V input of ESP32 and GPIO 16/17 is connected with the UART of Arduino. Three 18650 cells in parallel fed through a TP4056 charge/protection board whose output is boosted by an MT3608 to a stable 5.0V prior to feeding it into the Arduino and ESP32.



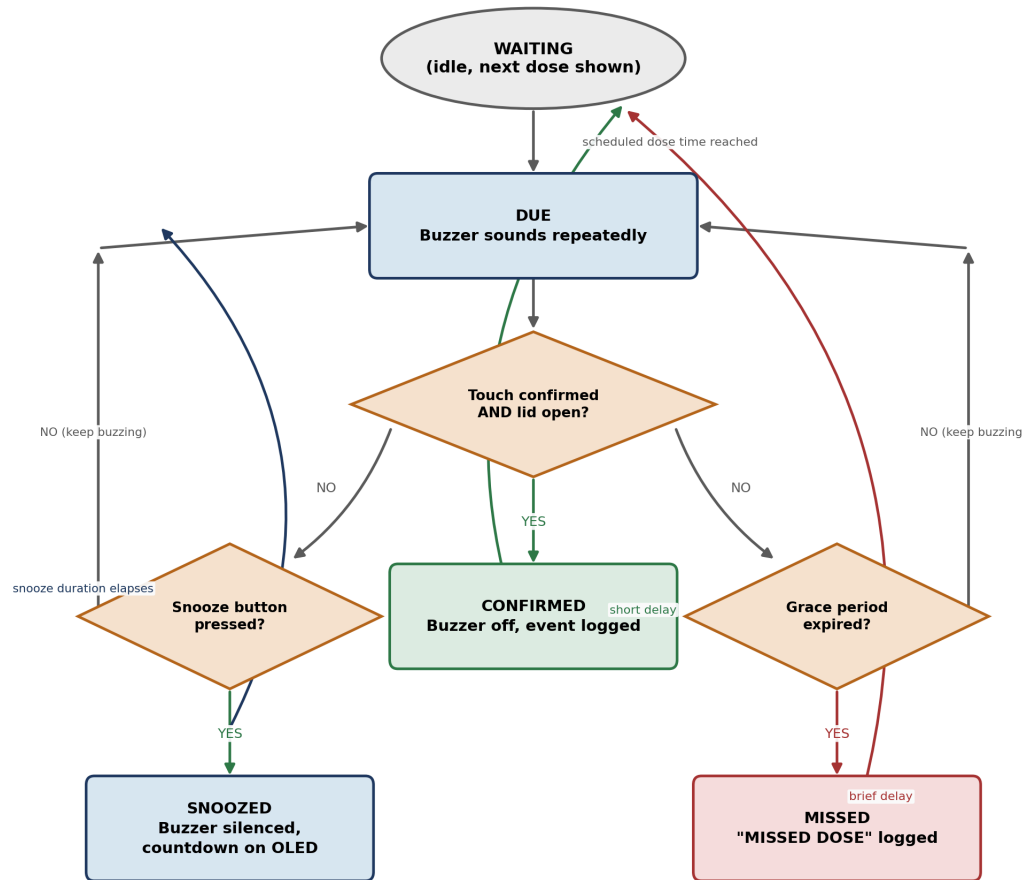
DoseFlow -- System Wiring / Block Diagram



**Implementation:** The touch and snooze buttons are checked continually within the main loop, the state of the lid is checked with both of the limit switches as well as the distance reading from the VL53L0X, serial packets are received and passed to the main loop, the adherence state machine is executed and the OLED is redrawn.

- **Waiting:** Current time is equal to scheduled dose time (Shows what time the current dose is scheduled and the next dose is scheduled; buzzer is not sounding).
- **Due:** Medicine Names appear on the OLED, enter from Waiting at scheduled time (Buzzer sounds in a repeating pattern, awaiting touch confirmation or snooze).
- **Snoozed:** Pressed snooze button; Due (Buzzer silenced for a configurable snooze duration; countdown displayed on OLED; resumes to Due at the end of snooze duration).
- **Confirmed:** Touch sensor pressed while lid is open (Buzzer silenced, “CONFIRMED” displayed, event recorded and synced to Cloud, light turns back to Waiting after a short time).
- **Missed:** No confirmation received before the grace period is up (“MISSED DOSE” appears; this event is logged and synced to cloud; returns to Waiting after a brief delay).

## DoseFlow -- Adherence Workflow Flowchart



In the ESP32, it first looks for any Wi-Fi information previously stored in the ESP32 memory. If it does not find any (or if the on-board BOOT button is held at boot time which clears the ESP32 memory), it sets up a local ESPAsyncWebServer captive portal access point (“Smart-Me Setup”) allowing the user to enter their network credentials and a snooze time via a simple web form. After connecting, it syncs time via NTP, passes the correct time to the Arduino, gets the day's schedule from the cloud application and enters its main loop which passes serial packets to and from the Arduino, re-syncs the schedule and any pending logs (every 30 seconds) and provides a live status dashboard page that displays the next dose, sync status, and a “Forget Network” reset button.

**Web Dashboard & Cloud Integration:** The production cloud application (dose.joysarkar.app) acts as the main control center with a robust set of features to monitor and motivate medication adherence with a paired Web interface.

- **Schedule & Medication Management:** Users can set up specific time periods (such as Morning, Afternoon, Evening) where medications are administered. It enables tracking of

different medicines and specific quantities of medicines (Dose Patterns) and allows real-time view of the next medicine scheduled (Next Schedule Banner) and the active/inactive capability for specific schedules without their deletion.

- **Extensive event logging:** Extensive event logging in the MongoDB database. It monitors normal actions (reminder times, user confirmations, missed doses), physical events (open/close dates of the lid) and records abnormalities (unscheduled access, sensor disagreement, "lid left open" alerts). It also records user actions such as snooze presses, and enables "Current Week" and "All Time" filtering of the logs.
- **Remote operation:** The physical smart box can be remotely configured using the dashboard through Device Configuration & Connectivity. This includes securely updating WiFi credentials, setting a snooze timer, defining an acceptable period of time window for a dose (e.g., 30 minutes), a lid warning timer for left open alerts and a real-time status monitor of the device's network connectivity.
- **User Profile & Security:** Secure authentication, user profile management with JWT and encrypted passwords. It also features an opt-in notification feature to alert you if you miss a dose or the box is opened unexpectedly.

**Challenges:** One of the most important design considerations was the ability to differentiate between a true open lid and noise generated by the mechanical limit switches as a loosely closed lid was moved against them: it is important to determine if the box is actually open and not just caused by noise on the mechanical limit switch so the design relies on both the state of the mechanical limit switch and on the distance reading signal from the VL53L0X. A second challenge was the safe communication of the two microcontrollers via UART, safely, with different logic level voltages (5V for the Arduino and 3.3V for the ESP32) without damaging the ESP32. Additionally, the firmware needed protection against double triggering; there are explicit booleans that determine whether a touch button has been pressed or not and whether a snooze button has been pressed or not, and a pair of Latch and Interrupt type flags for the touch and snooze button, respectively. Finally, even once again if Wi-Fi is intermittent, the events had to be buffered locally on the adherence log using the ESP32's 4MB on-board non-volatile flash (LittleFS) with a "synced" flag per event, so that none of these events are lost if the MongoDB backend is temporarily unavailable.

## 4. Sustainability & Impact

**Sustainability:** The device is powered by a rechargeable 3x 18650 Li-ion battery pack with TP4056 protection/charging circuit which eliminates the need for expended batteries that are often used in other devices. Arduino and ESP32 are both low power, re-usable devices that can be used again on other embedded projects - re-wiring is very simple (screw terminal) and they can be disassembled to be used in new projects without discards.

**Impact:** DoseFlow tackles a public health problem that is recognized for missed or mistimed medication doses affecting users of all ages, especially the elderly and those who are taking medications to manage chronic conditions. It provides a secure, low cost solution to remotely monitor adherence, as both patient and caregiver will be able to see all dose information and update it to a MongoDB database that may help reduce missed doses and ease the burden on the caregiver who would otherwise have to keep checking in.

**Future Work:** A companion app to the OLED (using React Native/Expo) to caregivers, to display adherence history and missed dose alerts in real-time, multiple compartments for multiple medications with individual reminders, and a battery level sensor displaying low battery alert on the OLED to warn the user that the battery is running low.

**Limitations:** It uses good WiFi connectivity for schedule updates and cloud logging, if no WiFi, the device will buffer events locally, but will not download the schedule. While the snooze is something users can choose to enable, it is done intentionally and therefore if a person doesn't respond within the time frame they will still see the dose as missed. If the ambient light is not sufficient because of the sensor's natural characteristics, or the sensor window is obstructed by the presence of a static object, the VL53L0X may also be affected by this, and the DS3231 won't be as accurate as it can be if it has not received an NTP resync for extended periods of time.

## 5. Results & Discussion

| Parameter                         | Value in Firmware                               | Purpose                                                                        |
|-----------------------------------|-------------------------------------------------|--------------------------------------------------------------------------------|
| Buzzer alert pattern (DUE state)  | 500 ms ON / 500 ms OFF (1 Hz)                   | Audible alarm while a dose is due                                              |
| Default snooze duration           | 10 minutes (configurable via ESP32 preferences) | Time before a snoozed reminder re-triggers as DUE                              |
| Grace period before MISSED        | 30 minutes from reminder onset                  | Window in which the user can still confirm before the dose is logged as missed |
| CONFIRMED display hold time       | 4 seconds                                       | How long "CONFIRMED!" is shown before returning to WAITING                     |
| MISSED display hold time          | 10 seconds                                      | How long "MISSED DOSE" is shown before returning to WAITING                    |
| VL53L0X ranging interval          | 250 ms continuous, 500 ms timeout               | Rate at which lid-open distance is re-checked                                  |
| OLED text scroll step             | Every 30 ms                                     | Scrolling speed of the medicine name on the display                            |
| ESP32 background sync interval    | Every 30 seconds                                | Re-fetch schedule and retry any unsynced logs                                  |
| Wi-Fi connect timeout (STA mode)  | Up to 10 seconds (20 × 500 ms retries)          | Time allowed to join the stored network before falling back to AP mode         |
| NTP time-sync timeout             | 10 seconds                                      | Time allowed to obtain real-world time before continuing without it            |
| Post-event schedule refresh delay | 3 seconds after a confirmed/missed log          | Brief pause before the ESP32 re-fetches the next schedule entry                |

These values are meant to be used to initialise a design that will ensure that locally, time critical loop will remain tight on sub-second time regardless of the network. The network dependent loop (WiFi join, NTP

sync, schedule/log sync with the cloud API) will be multi-second and it will be automatically restarted if it fails without blocking the Arduino state machine. In practice, once a dose is due the alert presented to the user (buzzer, OLED) should be "time of the hour" since it is only directly triggered by the local RTC comparison, it should be triggered instantly and need no time out to be triggered by any network round-trip only the cloud logging and the refresh of the schedule are subject to Wi-Fi latency and are designed to be eventually consistent, not real-time. The two most important direct time-to-beep delays will be: (1) the delay between the RTC's minute rollover and the buzzer's first audible beep which is at most a few hundred milliseconds. (2) the delay between a confirmed/missed event and the event being pushed into the MongoDB database which will be dominated by the Wi-Fi round-trip time and in the worst case, by the 30-second background sync interval if the immediate push attempt fails.

## 6. Conclusion

DoseFlow is an excellent combination of Arduino and ESP32 to make a connected medicine reminder box. Because that provides a physical confirmation of the dose compared to a mere timer. The system implemented here relies on I2C for local communication between sensors and displays, UART with level shifter for reliable communication between the Arduino and the ESP32 and HTTPS/REST for reliable synchronization of the ESP32 schedule. And logs with the cloud application and MongoDB backend with the increasing demand for drug adherence tools that can be verified and tracked remotely. Combines embedded hardware design and cloud-based software with a working example of concepts in computer interfacing. The completely rechargeable, affordable solution utilizing local caching via 4MB flash is designed to help minimize missed doses and provide helpful insights for caregivers to a patient's adherence while providing ample opportunities for further development to an app-based, multi-compartment product.

## 7. Contribution

| Name                 | ID       | Role(s) / Responsibilities    | Specific Contributions                                                                                                                                                                                                                                                                                                                                          |
|----------------------|----------|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Jotee Sarkar Joy     | 22301001 | Hardware Design & Prototyping | <ul style="list-style-type: none"> <li>Designed the dual-microcontroller (Arduino + ESP32) physical architecture.</li> <li>Implemented the TP4056 and MT3608 power management circuit for the 18650 battery pack.</li> <li>Wired the shared I2C bus, the level-shifted UART communication, and mounted the mechanical limit switches and ToF sensor.</li> </ul> |
| Subrina Jerrin Akter | 22201142 | Firmware Development          | <ul style="list-style-type: none"> <li>Programmed the core Arduino finite state machine (WAITING, DUE, SNOOZED, etc.) and OLED</li> </ul>                                                                                                                                                                                                                       |

|                             |          |                                             |                                                                                                                                                                                                                                                                                                                                          |
|-----------------------------|----------|---------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                             |          |                                             | UI. Integrated the VL53L0X and DS3231 sensor libraries. <ul style="list-style-type: none"> <li>Developed the ESP32 local captive portal, Wi-Fi handling, and LittleFS flash memory buffering system.</li> </ul>                                                                                                                          |
| Md. Abir Hasan Rohan        | 22101294 | Cloud Backend & Web Application             | <ul style="list-style-type: none"> <li>Designed the MongoDB database schema for schedules and adherence logs.</li> <li>Developed the REST API backend (including the `/logs`, `/schedules`, and `/acknowledge` endpoints).</li> <li>Built the user-facing web dashboard for remote device configuration and event monitoring.</li> </ul> |
| Shees Mohammad Hossain Saud | 20101234 | System Integration, Testing & Documentation | <ul style="list-style-type: none"> <li>Conducted end-to-end system testing (Wi-Fi latency, sensor disagreement edge cases, RTC power-loss).</li> <li>Calibrated the Time-of-Flight sensor distance thresholds.</li> <li>Designed the system flowcharts, block diagrams, and compiled the final technical report.</li> </ul>              |

## References

- Satwani, B. (2026). MediMind: A low-cost ESP32-based smart pill reminder system for elderly medication adherence. *International Journal of Engineering Research & Technology (IJERT)*, 15(7). <https://www.ijert.org>
- Peddisetti, V., Kandregula, P. K., John, J. A., Poomdla, S., George, K., & Panangadan, A. (2024). *Smart medication management: Enhancing medication adherence with an IoT-based pill dispenser and smart cup*. California State University, Fullerton. [https://www.fullerton.edu/ecs/faculty/apanangadan/publications/SmartMedicationManagement\\_A\\_IMHC-2024.pdf](https://www.fullerton.edu/ecs/faculty/apanangadan/publications/SmartMedicationManagement_A_IMHC-2024.pdf)
- World Health Organization. (2003). *Adherence to long-term therapies: Evidence for action*.